



King Salman International University

Faculty of Computer Science and Engineering

Project Title

Intelligent Robot for Flame Location Detection

A Thesis Submitted to Computer Engineering Department
Faculty of Computer Science & Engineering, KSIU

In Partial Fulfillment of **B.Sc.** Degree in Computer Engineering
Computer Engineering Program

By

Amin Ayssar Mubarak

Supervisors

Dr. Mahmoud Zaki

Assistant Professor, Faculty of Computer Science & Engineering
King Salman International University

EGYPT 2026

Abstract

This report details the system integration, backend API architecture, and asynchronous communication frameworks developed for an Intelligent Mobile Robot designed for autonomous flame location detection and suppression. To optimize power efficiency and maintain real-time responsiveness, the system employs distributed split-node architecture. While a stationary external unit acts as the global vision node utilizing custom Convolutional Neural Networks (CNNs), the embedded mobile edge node (Raspberry Pi 4 Model B) requires a robust integration layer to translate external neural inferences into localized ROS 2 navigation and hardware actuation safely.

The primary contribution of this work lies in the engineering of that integration layer. A localized Message Queuing Telemetry Transport (MQTT) communication pipeline was deployed utilizing an Eclipse Mosquitto broker, ensuring deterministic, near-zero latency data exchange while eliminating critical internet dependencies. To facilitate human-in-the-loop monitoring, a multi-threaded Python Flask microserver was engineered directly on the edge node. This backend serves a decoupled web interface, providing bidirectional control endpoints, dual-stream MJPEG video telemetry, and real-time hardware observability via `psutil`. Furthermore, a deterministic dual-layer software and hardware kill-switch mechanism was integrated into the API to enforce absolute immobility during active fire detection events. Integration testing confirmed that this decoupled architecture successfully eliminated procedural blocking, resulting in highly stable sub-millisecond safety responses required for hazardous environments.

Table of Contents

Abstract	3
1. Chapter 1: Introduction & Scope	7
1.1 Project Background	7
1.2 System Architecture Overview.....	7
1.3 Scope of System Integration.....	7
2. Chapter 2: Distributed Split-Node System Architecture.....	9
2.1 The Monolithic Bottleneck	9
2.2 The Split-Node Paradigm	9
2.3 The Global Vision Node (Perception Layer)	9
2.4 The Mobile Edge Node (Control Layer)	10
3. Chapter 3: Embedded Platform and Sensor Integration	11
3.1 Core Processing Hardware: Raspberry Pi 4 Model B	11
3.2 Global Surveillance Sensor: TP-Link Tapo C210	11
3.3 Local Telemetry Sensor: Pi Camera Module V3	11
3.4 Actuation Drivers: Dual BTS7960 H-Bridges	12
4. Chapter 4: MQTT Communication Layer	13
4.1 The Role of Asynchronous Messaging	13
4.2 Architectural Migration: Cloud vs. Edge Deployment.....	13
4.3 Topic Schema and Architectural Decoupling	14
4.4 Security and Startup Dependencies	16
Chapter 5: RESTful API and Telemetry Microserver	17
5.1 Framework Selection: Why Flask for Embedded Edge APIs?	17
5.2 RESTful Endpoint Architecture	17
5.3 Video Encoding and Multipart Streaming Pipeline	18
5.4 Cross-Origin Resource Sharing (CORS) and Network Access	18
Chapter 6: Hardware Observability Backend	19

6.1 The Need for Hardware Observability.....	19
6.2 Real-Time Polling Architecture	19
6.3 Telemetry Data Structure and Endpoint Integration	20
Chapter 7: Multi-Threading Architecture	20
7.1 The Synchronous Execution Bottleneck	20
7.2 Daemon Thread Decoupling.....	20
7.3 Inter-Thread Communication and State Management	21
Chapter 8: Kill-Switch Safety Loop.....	22
8.1 Deterministic Safety Protocols in Autonomous Systems	22
8.2 Software Layer Enforcement and Payload Parsing	22
8.3 Hardware Layer Isolation (PWM Severing)	23
Chapter 9: AI-to-Control Integration Pipeline.....	24
9.1 The End-to-End Event Chain	24
9.2 Stage 1: Perception and Payload Generation	24
9.3 Stage 2: Interception and Safety Halting.....	24
9.4 Stage 3: Autonomous Actuation Handoff	25
Chapter 10: Integration Challenges & Debugging.....	26
10.1 The Complexity of Systems Integration.....	26
10.2 Laptop Node: Visual Telemetry Buffer Saturation	26
10.3 Edge Node: GPIO Concurrency and Race Conditions	26
10.4 Network Layer: Intermittent Cloud Latency	27
Chapter 11: Verification & Results	28
11.1 Integration Testing Methodology	28
11.2 Network Latency and Broker Stability Results	28
11.3 Multi-Threading and Video Telemetry Performance	28
11.4 Kill-Switch Determinism	29
Chapter 12: Limitations & Future Work	30
12.1 Current System Limitations.....	30

12.2 Future Architectural Enhancements	31
Chapter 13: Conclusion.....	32
13.1 Project Summary and Architectural Success	32
13.2 Key Engineering Achievements	32
13.3 Final Remarks	33
Reference.....	34
Appendix A: MQTT Topic & JSON Payload Schemas.....	35
Appendix B: Edge Node Hardware Pin Mapping	36

1. Chapter 1: Introduction & Scope

1.1 Project Background

Fire hazards within industrial environments, multi-level warehouses, and petrochemical facilities present catastrophic risks to human life, high-value physical inventory, and structural integrity. Traditional stationary fire protection frameworks, such as localized smoke alarms and ceiling-mounted sprinkler networks, frequently suffer from spatial limitations, latency in activation, and severe collateral water damage. To mitigate these deficiencies, autonomous mobile robotics has emerged as a crucial paradigm shift. By deploying targeted, real-time fire suppression vectors, autonomous systems remove human personnel from highly toxic and structurally compromised containment zones.

Project "Phoenix", developed at King Salman International University (KSIU), addresses this critical gap through the development of an Intelligent Mobile Robot engineered for autonomous flame location detection, life-safety asset prioritization, and localized fire extinguishing.

1.2 System Architecture Overview

The overarching systemic framework of Project Phoenix utilizes a distributed, split-node edge computing topography. To balance computational load, power efficiency, and real-time responsiveness, the architecture segregates high-intensity computer vision tasks from onboard navigation and actuation.

A stationary global vision station (the Laptop Node) monitors the operational environment via a wide-area surveillance camera, executing custom Convolutional Neural Networks (CNNs) to localize fire boundaries and map human targets actively in danger. Real-time coordinates and alerts are packaged into JSON payloads and streamed asynchronously to the mobile edge computing node (a Raspberry Pi 4 Model B). This specialized pipeline ensures that high-intensity neural computing is safely decoupled from the physical platform, reducing mobile thermal loading and battery consumption while enabling rapid navigational path replanning.

1.3 Scope of System Integration

While Project Phoenix represents a highly interdisciplinary engineering effort—encompassing kinematics, mechanical design, and artificial intelligence, this thesis explicitly focuses on the backend system integration, distributed communication architecture, and teleoperation API managed strictly by the author.

To bridge the gap between high-level perception and low-level hardware actuation, a robust software integration layer was required. The core scope of this independent research comprises the design, implementation, and optimization of that backend infrastructure. Specifically, the author's contributions encompass:

- **RESTful API and Telemetry Microserver:** Development of a multi-threaded Python Flask backend hosted on the embedded edge node. This microserver handles synchronous HTTP requests for manual hardware override while managing an asynchronous, dual-stream MJPEG video telemetry pipeline for remote human-in-the-loop monitoring.
- **Hardware Observability Backend:** Implementation of a real-time, thread-safe hardware polling pipeline utilizing `psutil` and `vcgencmd` to monitor CPU utilization, thermal states, and memory allocation, dynamically preventing computational throttling during mission-critical deployments.
- **MQTT Communication Layer Integration:** Design and deployment of the publish/subscribe messaging infrastructure. This involved a critical architectural migration from a cloud-dependent broker to a fully localized Mosquitto edge broker, achieving sub-millisecond latency for safety-critical data transfer.
- **Kill-Switch Safety Loop:** Engineering a deterministic, dual-layer (software and hardware) fail-safe mechanism. This integration directly manipulates internal control flags and intercepts Pulse-Width Modulation (PWM) duty cycles to ensure the robot remains safely stationary during an active fire suppression event.
- **Multi-Threading Architecture:** Resolution of severe thread-blocking bottlenecks by decoupling the camera capture, network listening, HTTP request handling, and General-Purpose Input/Output (GPIO) execution into independent, non-blocking daemon threads.

2. Chapter 2: Distributed Split-Node System Architecture

2.1 The Monolithic Bottleneck

A primary constraint in the design of autonomous mobile robotics is the inverse relationship between computational density and onboard power availability. Initial conceptual outlines for Project Phoenix considered a monolithic architecture—deploying the deep learning computer vision models, the web server, and the autonomous navigation stack directly onto the onboard Raspberry Pi 4 Model B.

However, system profiling rapidly revealed that this was unviable. Executing concurrent Convolutional Neural Networks (CNNs) for fire and human detection alongside the ROS 2 Nav2 costmap generators exceeded the embedded system's hardware limits. It induced severe thermal throttling, caused frame-drops in the video telemetry, and ultimately led to catastrophic desynchronization of the control loops.

2.2 The Split-Node Paradigm

To circumvent these hardware limitations without compromising the intelligence of the system, a distributed, split-node edge computing topography was engineered. This architecture explicitly decouples high-intensity perception from low-level hardware actuation, dividing the computational load across two distinct physical machines connected via a strictly local, low-latency Wi-Fi subnet. This division of labor forms the foundational architecture of Project Phoenix and directly necessitated the robust integration layer developed in this thesis.

2.3 The Global Vision Node (Perception Layer)

The first half of the architecture is the stationary Global Vision Node, hosted on a dedicated high-performance laptop. This node serves as the overarching intelligence hub for the operational environment. It ingests continuous Real-Time Streaming Protocol (RTSP) video feeds from a wide-area Tapo C210 surveillance camera mounted above the hazard zone.

Utilizing its superior processor capabilities, the Vision Node continuously executes the custom GFD-Net fire detection and Keras human detection algorithms. Rather than attempting to send heavy, processed video frames to the robot, this node acts as a data refiner. It extracts only the mathematically significant state data—spatial coordinates,

confidence floats, and Boolean safety triggers—packages them into lightweight JSON payloads and publishes them to the network.

2.4 The Mobile Edge Node (Control Layer)

The second half of the architecture is the Mobile Edge Node, physically embodied by the Raspberry Pi 4 mounted on the robotic chassis. By actively stripping the burden of deep neural network processing from the onboard computer, the Pi can dedicate its processing cycles entirely to real-time, deterministic operations.

The Mobile Edge Node acts as an intelligent actuator. It hosts the local Mosquitto communication broker, the Flask microserver API, and the ROS 2 navigation routines. Its sole architectural purpose is to asynchronously consume the lightweight JSON payloads published by the Vision Node, verify internal safety states via the multi-threaded kill-switch, and convert mathematical targets into precise mechanical movements via Pulse-Width Modulation (PWM).

3. Chapter 3: Embedded Platform and Sensor Integration

3.1 Core Processing Hardware: Raspberry Pi 4 Model B

To serve as the Mobile Edge Node, the Raspberry Pi 4 Model B (4GB RAM variant) was selected as the central embedded computer. The selection of this specific System-on-a-Chip (SoC) was driven by the strict concurrency requirements of the integration layer.

Powered by a Broadcom BCM2711 Quad-core Cortex-A72 processor, the Pi 4 possesses the multi-threading capabilities necessary to simultaneously host the Mosquitto communication broker, execute the Flask microserver, and run the ROS 2 navigation stack. Furthermore, its native 40-pin General-Purpose Input/Output (GPIO) header provides the indispensable low-level hardware interface required to generate the Pulse-Width Modulation (PWM) signals that drive the robot's locomotion and pump mechanisms.

3.2 Global Surveillance Sensor: TP-Link Tapo C210

For the perception layer to successfully monitor the operational environment, a high-fidelity, wide-area sensor was required. The Tapo C210 Pan/Tilt Wi-Fi camera was selected as the global surveillance node.

Mounted in a fixed, elevated position above the hazard zone, this sensor captures 3-Megapixel high-definition video. Crucially for the software integration architecture, the Tapo C210 natively supports the Real-Time Streaming Protocol (RTSP). This allowed the stationary Laptop Vision Node to securely ingest the live video feed over the local area network using OpenCV, processing the frames through the GFD-Net neural network without relying on direct USB or physical cable tethering to the robot.

3.3 Local Telemetry Sensor: Pi Camera Module V3

While the global camera provides structural mapping, remote operators require a first-person perspective to verify fire alignment prior to manual suppression. The Pi Camera Module V3, featuring a 12-Megapixel Sony IMX708 sensor with powered autofocus, was integrated directly onto the robotic chassis.

Interfacing with the Pi 4 via the high-speed MIPI Camera Serial Interface (CSI-2) port, this module completely bypasses the Pi's USB bus, conserving critical bandwidth. The raw frames from this sensor are captured by the backend Flask integration script, dynamically

compressed into JPEG byte-arrays, and streamed via the asynchronous multipart/x-mixed-replace pipeline detailed in Chapter 5.

3.4 Actuation Drivers: Dual BTS7960 H-Bridges

To translate the digital software commands into physical locomotion, the system utilizes dual BTS7960 high-current motor driver modules. These solid-state H-bridges are capable of handling up to 43 Amperes of current, which is necessary to safely drive the heavy-duty 24V DC geared motors responsible for moving the robotic chassis.

From a systems integration perspective, these drivers serve as the critical endpoint for the hardware safety loop. By wiring the BTS7960 enable pins (R_EN and L_EN) directly to the Pi's GPIO pins, the backend Python script maintains absolute control over the electrical flow. When the robot/fire_detected payload triggers the integration layer's kill-switch, the software zeroes out the logic-level PWM signals. This instantly collapses the H-bridge, electrically isolating the 24V motor supply and guaranteeing deterministic immobility during fire suppression.

4. Chapter 4: MQTT Communication Layer

4.1 The Role of Asynchronous Messaging

A fundamental engineering challenge in the Project Phoenix architecture was establishing a reliable, low-latency communication bridge between the computationally heavy Laptop Vision Node and the embedded Raspberry Pi Mobile Edge Node. Because the neural network inference cycles run at variable frequencies depending on frame complexity, a synchronous communication protocol (such as standard HTTP REST polling) would cause blocking execution and severe data bottlenecks.

To resolve this, the Message Queuing Telemetry Transport (MQTT) protocol was selected. MQTT is an asynchronous, publish/subscribe messaging protocol designed explicitly for constrained devices and low-bandwidth, high-latency networks. By utilizing the `paho-mqtt` Python client library, the system was decoupled: the Vision Node publishes coordinate payloads the moment they are calculated, and the Edge Node consumes them asynchronously without blocking its internal motor execution loops. Furthermore, MQTT was chosen over native ROS 2 Data Distribution Service (DDS) due to multi-cast firewall restrictions commonly enforced on university and industrial local area networks.

4.2 Architectural Migration: Cloud vs. Edge Deployment

During the initial prototyping phase, HiveMQ Cloud was utilized as the centralized broker. While this allowed for rapid initial deployment, rigorous integration testing exposed critical vulnerabilities in utilizing a cloud-based broker for a safety-critical robotics platform.

The primary vulnerability was a strict dependency on external internet access. If the warehouse Wi-Fi lost its connection to the wider internet, the robot would immediately lose its ability to receive emergency kill-switch commands or fire coordinates, rendering the autonomous system blind and uncontrollable. Additionally, routing messages through an external cloud server introduced a network round-trip latency of 50 to 200 milliseconds. In a dynamic fire suppression scenario, this latency delay between visual detection and physical halting was deemed unacceptable.

To achieve true deterministic safety, a major architectural migration was engineered. The cloud dependency was entirely removed by deploying an Eclipse Mosquitto broker directly onto the Raspberry Pi 4 edge node. This localized deployment yielded significant architectural advantages:

1. **Absolute Internet Independence:** The robotic platform achieved fully localized autonomy. The Vision Node and the Edge Node now communicate exclusively over the local subnet, allowing operation in electromagnetically shielded or remote industrial environments.
2. **Near-Zero Latency:** By routing packets internally within the local network (and entirely via loopback localhost for internal edge node communications), message latency was reduced to sub-millisecond ranges, ensuring instantaneous kill-switch activation.
3. **Deterministic Fault Tolerance:** Removing the external cloud server eliminated a major single point of failure (SPOF) from the control loop pipeline.

4.3 Topic Schema and Architectural Decoupling

The backend communication architecture was structured around a strictly defined topic schema. A critical design decision was the explicit decoupling of navigational data from safety-critical control signals.

Rather than reusing the `ambers/robot/navigation/target` topic (which carries 3D float coordinates to the ROS 2 Nav2 stack) to also trigger the motor kill-switch, a dedicated Boolean safety topic (`robot/fire_detected`) was engineered. This separation of concerns provides three distinct advantages:

1. **Independent Failure Resilience:** If the Nav2 coordination node crashes or coordinate parsing fails, the safety kill-switch still fires independently via its own dedicated subscriber pipeline.
2. **Lightweight Parsing:** The Flask subscriber only parses a minimal JSON payload containing a Boolean state and a confidence float, avoiding the computational overhead of continuous coordinate math on the safety-critical path.
3. **Non-Intrusive Integration:** Bridging the Laptop Vision Node to this safety layer required adding only a single parallel publish command to the external Computer Vision code, ensuring zero disruption to the existing navigation codebase.

Similarly, a parallel topic (`robot/human_detected`) was established to handle life-safety priority overrides without tangling the logic of structural fire suppression.

Table 4.1: MQTT Topic Definitions and Payload Structures

Topic	Publisher	Subscriber	Payload Format	Purpose
robot/fire_detected	Vision Node	Edge Node (Integration Backend)	JSON {fire: bool, confidence: float, timestamp: string}	Triggers the physical hardware kill-switch.
robot/human_detected	Vision Node	Edge Node (Integration Backend)	JSON {human: bool, confidence: float, timestamp: string}	Triggers priority life-safety override alerts.
ambers/robot/navigation/target	Vision Node	Edge Node (ROS2 Nav2 Stack)	JSON {x: float, y: float}	Dispatches target spatial coordinates to Nav2.
robot/control/enable	Edge Node	Motor Logic	Boolean	Internal software safety gating.
robot/status	Edge Node	Web Dashboard	JSON (Full Status Object)	Telemetry data for remote observability.

4.4 Security and Startup Dependencies

For local laboratory deployment, the Mosquitto broker was configured to handle anonymous internal routing to facilitate rapid node testing. However, the architecture was designed with futureproofing in mind. The broker configuration (`mosquitto.conf`) supports immediate transition to authenticated connections using `mosquitto_passwd` and TLS certificates (`ca.crt`, `server.crt`) to prevent unauthorized network hijacking in a production environment.

Because the MQTT broker acts as the central nervous system of the robot, strict startup dependencies were enforced. The Mosquitto service is configured to initialize at the operating system level prior to any Python scripts or ROS 2 nodes. The custom `mqtt_nav_client.py` script requires a 5-second initialization delay to ensure the broker and the ROS 2 Action Server are fully responsive before attempting to bind socket connections, preventing race conditions during system boot.

5. Chapter 5: RESTful API and Telemetry Microserver

5.1 Framework Selection: Why Flask for Embedded Edge APIs?

To provide human operators with manual control capabilities and systemic oversight, a centralized backend server was required on the Mobile Edge Node. Python's Flask framework was selected as the foundational architecture for this microserver.

The selection of Flask over heavier frameworks like Django or asynchronous frameworks like FastAPI was a highly deliberate engineering decision based on hardware constraints. The Raspberry Pi 4 possesses limited RAM and CPU cycles, the majority of which must be reserved for the ROS 2 navigation stack. Flask provides a minimalistic, low-overhead WSGI (Web Server Gateway Interface) web application setup. Furthermore, its synchronous routing capabilities integrate seamlessly with the native RPi.GPIO and gpiozero libraries, allowing HTTP web requests to be translated directly into low-level hardware actuation without complex middleware.

5.2 RESTful Endpoint Architecture

The microserver exposes a series of RESTful endpoints that serve as the primary programmatic interface for the decoupled frontend Graphical User Interface (GUI). By utilizing the POST method for state-changing actions and the GET method for telemetry retrieval, the API adheres to standard REST architecture principles.

The core movement routes (`/move/forward`, `/move/backward`, `/move/left`, `/move/right`) do not directly control the motor pins. Instead, they act as safe data enqueueers. When an operator triggers a movement POST request, the backend route intercepts the command and immediately evaluates the global `fire_detected_flag`. If the flag is `FALSE`, the backend publishes the corresponding velocity command to the `/cmd_vel` ROS 2 topic, which the dedicated motor controller node consumes. If the flag is `TRUE`, the API categorically rejects the request with an HTTP 403 Forbidden status, enforcing the software kill-switch at the network level.

Additionally, a high priority `/kill` endpoint is exposed. This route allows the remote operator to manually engage the emergency stop protocol, halting all PWM generation overriding any active navigational goals.

5.3 Video Encoding and Multipart Streaming Pipeline

Providing real-time situational awareness via video telemetry was a critical requirement. However, standard video streaming protocols (like H.264 over WebRTC) require significant CPU encoding overhead, which would induce thermal throttling on the edge node.

To solve this, a lightweight Motion JPEG (MJPEG) streaming pipeline was engineered. The backend captures raw matrix frames from both the Pi Camera V3 and the external Vision Node's processed overlay feed. These raw matrices are rapidly compressed into JPEG byte arrays using OpenCV (`cv2.imencode`).

The Flask application then utilizes a generator function to serve these byte arrays continuously using the `multipart/x-mixed-replace` MIME type. This specific HTTP header instructs the client's web browser to continually replace the current image with the next incoming JPEG frame. This pipeline is exposed via two distinct GET endpoints (`/video_feed` and `/video_feed_tapo`), allowing the decoupled frontend dashboard to render simultaneous, low-latency video streams strictly through standard HTTP DOM elements without requiring third-party video plugins.

5.4 Cross-Origin Resource Sharing (CORS) and Network Access

Because the architecture relies on a decoupled frontend, the HTML/JS dashboard often runs on a different port or a completely different physical machine (the operator's laptop) than the API server running on the robot (the Pi). Modern web browsers block these types of cross-network API requests by default for security reasons.

To facilitate seamless teleoperation, the Flask backend was configured with explicit Cross-Origin Resource Sharing (CORS) headers. By utilizing the `flask_cors` library, the microserver explicitly whitelists incoming pre-flight OPTIONS requests from the local subnet. This architectural decision ensures that the human operator can access the control endpoints and video feeds securely across the university or industrial local area network.

6. Chapter 6: Hardware Observability Backend

6.1 The Need for Hardware Observability

Embedded edge computing nodes operating within autonomous robotic frameworks are highly susceptible to thermal and computational degradation. The Raspberry Pi 4 Model B, while capable of running dense ROS 2 navigational graphs and simultaneous video encoding pipelines, suffers from severe thermal throttling when core processor temperatures approach 80°C.

If thermal throttling occurs silently during a hazardous deployment—such as navigating a high-temperature warehouse fire environment—the internal control loops will desynchronize. This desynchronization manifests as dropped network frames, delayed emergency stops responses, and erratic physical navigation. Therefore, proactive hardware observability is not merely a diagnostic feature, but a critical safety requirement. To prevent mission failure, the human operator must have real-time visibility into the edge node's thermal and computational health prior to engaging autonomous suppression routines.

6.2 Real-Time Polling Architecture

To pre-emptively detect hardware stress without blocking the primary application logic, a robust hardware polling pipeline was engineered. A dedicated `SystemMonitor` backend class was implemented to run continuously as a background daemon thread. By isolating this process, the system can continuously sample hardware states without interrupting the rapid execution cycles required for motor PWM generation or incoming MQTT message listening.

The polling engine relies on two primary data extraction methodologies:

4. **System Resources:** The `psutil` Python library is utilized to extract precise, cross-platform metrics regarding CPU utilization percentages, RAM allocation (Used vs. Total Megabytes), and Disk Space availability.
5. **Thermal Metrics:** Because native Python libraries cannot securely access the Broadcom CPU thermal registers directly on the Raspberry Pi OS, the polling thread executes a secure OS-level subprocess call using the `vcgencmd measure_temp` command. This extracts the exact core temperature from `thermal_zone0`.

To prevent race conditions—where the background polling thread attempts to write hardware data to memory at the exact same millisecond the Flask server attempts to read

it for a web request—a strict `threading.Lock()` object was implemented. This ensures thread-safe memory reads and guarantees data integrity.

6.3 Telemetry Data Structure and Endpoint Integration

The `SystemMonitor` thread polls these critical hardware states at a fixed interval of every two seconds. Once collected, the data is aggregated alongside real-time sensor health flags (`pi_camera_capturing` and `rtsp_stream_alive`) and system uptime metrics.

This aggregated data is formatted into a strictly defined JSON dictionary (see Appendix A for the complete schema). The Flask microserver exposes this data matrix via the synchronous `/status` GET endpoint. When the decoupled frontend dashboard requests a status update, the backend instantly returns this thread-safe JSON payload. This architecture allows the operator to constantly monitor computational stability, providing early warnings for camera failures or impending thermal shutdowns before they compromise the robotic platform's safety parameters.

7. Chapter 7: Multi-Threading Architecture

7.1 The Synchronous Execution Bottleneck

During the initial phases of software integration, the embedded control script relied on a standard synchronous execution model. The Pi Camera capture routines, the `client.loop_forever()` MQTT listener, the Flask HTTP request handlers, and the GPIO motor execution commands were all executed sequentially within a single main Python thread.

This architecture introduced a severe, mission-critical bottleneck: procedural blocking. In Python, synchronous execution dictates that if one operation halts to wait for Input/Output (I/O), the entire program freezes. Consequently, if the camera module dropped a frame, or if a slow client requested the web dashboard, the MQTT listener was temporarily blinded. This resulted in frozen visual telemetry feeds and, most dangerously, delayed emergency stops responses since the software could not process the `fire_detected` payload while busy serving a web page.

7.2 Daemon Thread Decoupling

To resolve this architectural flaw without rewriting the entire embedded library ecosystem to support asynchronous coroutines (`async/await`), a concurrent multi-threading architecture was engineered using Python's native `threading` module.

The core system logic was decoupled and distributed across four independent, time-critical daemon threads. By setting `daemon=True`, these threads operate continuously in the background and automatically terminate when the main program exits, preventing zombie processes from consuming the Pi's memory.

The four isolated execution contexts are:

1. **Capture Thread:** Dedicated exclusively to pulling frames from the Pi Camera V3 into memory. By isolating this, camera I/O latency can no longer interrupt network listening.
2. **MQTT Thread:** A continuous background loop (`client.loop_start()`) dedicated solely to monitoring incoming network payloads, guaranteeing sub-millisecond interception of fire alerts.
3. **Flask Server Thread:** Handles asynchronous HTTP web requests from remote operators without pausing hardware execution.
4. **Motor Control Thread:** Isolates strict GPIO PWM generation, ensuring continuous signal delivery to the BTS7960 drivers.

7.3 Inter-Thread Communication and State Management

Because multiple independent threads must read and write to the same system variables (e.g., the Flask thread receiving a "Stop" command and the Motor thread executing it), memory race conditions posed a significant risk.

To manage inter-thread communication safely, global state flags were implemented alongside thread-safe data structures. All direct hardware access (modifying GPIO pins) was strictly localized to the Motor Control Thread. The Flask thread acts purely as an interface, modifying a shared global velocity variable. The Motor Control Thread continuously reads this shared variable at a fixed Hertz rate and updates the physical PWM duty cycles accordingly. This isolation guarantees that a crashed web request will never leave a hardware pin stuck in a "HIGH" voltage state, fundamentally stabilizing the robotic control loop.

8. Chapter 8: Kill-Switch Safety Loop

8.1 Deterministic Safety Protocols in Autonomous Systems

A paramount operational requirement for the Project Phoenix robotic platform is deterministic state management during hazardous deployments. Specifically, the robot must remain absolutely stationary while actively dispensing fire suppression agents. Any unpredictable locomotion during a suppression event could inadvertently spread the thermal hazard or induce catastrophic damage to the robot's localized pump mechanics.

To guarantee this immobility, a highly fault-tolerant kill-switch architecture was required. Rather than relying on a single point of logic, a dual-layer safety protocol (Software Enforcement and Hardware Isolation) was integrated into the core backend application, ensuring the drivetrain is disabled instantaneously upon visual confirmation of a fire.

8.2 Software Layer Enforcement and Payload Parsing

The first layer of the fail-safe mechanism operates at the network and application level, triggered by the dedicated robot/fire_detected MQTT topic. The integration event chain executes as follows:

1. **Detection & Publication:** Upon detecting a fire anomaly, the Laptop Vision Node (GFD-Net) constructs a JSON payload containing the Boolean state, the detection confidence float, and an ISO8601 timestamp, publishing it to the local Mosquitto broker.
2. **Interception & Verification:** The dedicated Flask MQTT subscriber thread intercepts the payload. The backend parses the JSON and verifies two conditions: `payload["fire"] == True AND payload["confidence"] >= [PLACEHOLDER-1: Confidence Threshold]`.
3. **Flag Assertion:** If verified, the thread synchronously asserts a global, thread-safe variable: the `fire_detected_flag`.
4. **API Gating:** Once this flag is active, the Flask HTTP layer immediately alters its routing logic. Any incoming POST requests targeted at the `/move/*` endpoints from a remote operator are intercepted and rejected with an HTTP 403 Forbidden status (`{"status": "blocked", "reason": "fire_detected_active"}`).

The software layer places a digital quarantine on the motor commands. This state persists, and is displayed prominently on the operator dashboard, until [PLACEHOLDER-2: Fire-

cleared behavior—either manual /kill reset or automatic {"fire": false} payload from the Vision Node].

8.3 Hardware Layer Isolation (PWM Severing)

While software flags provide robust routing control, relying solely on application-layer logic in a safety-critical system represents an engineering vulnerability. Unhandled software exceptions, memory leaks, or thread crashes within the Python interpreter could theoretically bypass the logical gates, leaving the robot stuck in a continuous movement loop.

To mitigate this, a secondary hardware layer was integrated directly into the RPi.GPIO execution thread. The moment the `fire_detected_flag` is asserted, the backend integration script executes an emergency override function that drops the Pulse-Width Modulation (PWM) duty cycles of the targeted Broadcom GPIO pins (Pins 11, 13, 16, and 22) to exactly 0%.

By severing the PWM signal entirely, the internal H-bridge channels of the physical dual BTS7960 motor drivers are electronically disabled. This hardware-level isolation ensures that no electrical current can flow to the 24V DC geared locomotion motors. Consequently, even if a contradictory software command is erroneously generated or the Flask server crashes entirely, the robot is physically immobilized by the lack of logic-level voltage, achieving true deterministic safety.

9. Chapter 9: AI-to-Control Integration Pipeline

9.1 The End-to-End Event Chain

The culmination of the software integration architecture is the seamless execution of the AI-to-Control pipeline. For the Project Phoenix mobile robot to function autonomously, high-level artificial intelligence inferences generated on an external machine must be deterministically translated into physical, localized actuation. The integration pipeline accomplishes this by managing a strictly timed, multi-stage event chain spanning perception, transmission, safety interception, and autonomous handoff.

9.2 Stage 1: Perception and Payload Generation

The event chain initiates at the Laptop Vision Node. The global surveillance sensor (Tapo C210) captures an RTSP video frame of the environment. The custom GFD-Net Convolutional Neural Network processes this frame, returning a positive fire classification alongside a bounding box.

Immediately, the Vision Node's coordinate transformation script utilizes an ArUco fiducial marker PnP (Perspective-n-Point) solver to map the 2D bounding box center into 3D physical workspace coordinates. To ensure architectural decoupling (as detailed in Chapter 4), the laptop publishes two simultaneous, independent JSON payloads to the local Mosquitto broker:

1. A safety payload published to `robot/fire_detected` containing the Boolean state and detection confidence.
2. A navigation payload published to `ambers/robot/navigation/target` containing the x and y float coordinates.

9.3 Stage 2: Interception and Safety Halting

Because local network latency via the Mosquitto broker is strictly sub-millisecond, the Raspberry Pi Mobile Edge Node intercepts these payloads almost instantaneously.

The dedicated MQTT background thread on the Pi receives the `robot/fire_detected` payload first. It parses the JSON, verifies the confidence threshold, and immediately asserts the `fire_detected_flag`. This triggers the dual-layer kill-switch engineered within the Flask microserver integration layer. All incoming teleoperation web requests are forcibly rejected, and the integration script drops the duty cycles of the Broadcom motor

control pins (GPIO 17, 27, 23, and 25) to 0%. The robot is now physically immobilized and secured against operator error.

9.4 Stage 3: Autonomous Actuation Handoff

Simultaneously, while the safety layer actively locks out manual teleoperation, the navigation payload (ambers/robot/navigation/target) is consumed by the dedicated ROS 2 Action Client running in parallel on the Pi.

This action client translates the JSON coordinates into a standard `nav2_msgs/action/NavigateToPose` goal, dispatching it to the ROS 2 Nav2 stack. The Nav2 stack takes control of the drivetrain, overriding the manual teleoperation lockout to autonomously pilot the robot to the fire source using LiDAR costmaps.

Once the robot arrives at the precise x and y coordinates, the Nav2 stack reports a `SUCCEEDED` status code. This triggers an internal `target_reached` Boolean flag. Finally, a dedicated ROS 2 pump controller node reads this flag and asserts a HIGH signal to GPIO 26, triggering the opto-isolated relay to engage the 24V diaphragm pump for a five-second suppression burst. Through this pipeline, a visual anomaly processed on a laptop successfully results in physical water displacement on the edge node.

10. Chapter 10: Integration Challenges & Debugging

10.1 The Complexity of Systems Integration

Integrating discrete subsystems—specifically high-level computer vision algorithms, asynchronous network protocols, and low-level hardware actuation—introduces complex points of failure. During the development of the Project Phoenix backend architecture, several critical bugs and performance bottlenecks were encountered. These challenges were primarily rooted in memory management, thread concurrency, and network latency. Resolving them required systematic debugging and architectural refactoring.

10.2 Laptop Node: Visual Telemetry Buffer Saturation

One of the most significant challenges encountered on the stationary Laptop Vision Node was severe latency in the MJPEG video stream sent to the remote operator dashboard. During deep learning inference cycles, the custom GFD-Net model required significant processing time per frame. Because the OpenCV VideoCapture object reads from a hardware-level First-In-First-Out (FIFO) buffer, the slow inference loop caused unread frames to stack up in the queue.

When the stream was viewed on the Flask dashboard, this buffer saturation manifested as "stale" visual telemetry—the operator was seeing events that had occurred several seconds in the past.

Solution (The Frame Drainage Engine): To resolve this without decreasing the camera's hardware framerate, a custom "Frame Drainage Engine" was engineered. A dedicated background thread was deployed to continuously call the `read()` function as fast as the hardware allowed, actively discarding old frames and saving only the absolute latest frame to a shared memory reference. The main AI inference loop and the Flask streaming route were refactored to pull exclusively from this memory reference rather than the hardware buffer. This decoupled architecture successfully eliminated stream lag, reducing visual latency to under 10 milliseconds.

10.3 Edge Node: GPIO Concurrency and Race Conditions

On the Raspberry Pi Mobile Edge Node, initial integration testing resulted in sporadic system crashes and erratic motor behavior. Debugging via system logs revealed that the `RPi.GPIO` library was throwing concurrency exceptions.

This occurred because the Flask web application and the automated safety routines were both attempting to alter the state of the physical motor pins simultaneously. If a remote operator pressed the "Forward" button at the exact millisecond the MQTT thread triggered a kill-switch halt, a memory race condition occurred over the physical GPIO registers, crashing the Python interpreter.

Solution (Hardware Access Isolation): The solution required strict structural refactoring of hardware ownership. Direct manipulation of the GPIO pins was entirely stripped from the Flask route handlers and the MQTT subscriber thread. Instead, a dedicated Motor Control Thread was established as the sole owner of the physical pins. The web routes and safety loops were rewritten to act purely as data enqueueers—they update global, thread-safe state variables. Motor Control Thread reads these variables at a fixed Hertz rate and executes the physical PWM changes synchronously, completely eliminating race conditions.

10.4 Network Layer: Intermittent Cloud Latency

As noted in Chapter 4, the initial network architecture relied on HiveMQ Cloud. During integration testing, the kill-switch response time was highly erratic. A fire detection event would sometimes halt the robot in 50 milliseconds, and other times take upwards of 400 milliseconds.

Network packet sniffing and latency profiling revealed that routing local data through a remote cloud server subjected the safety loop to unpredictable internet traffic bottlenecks and ISP throttling. In a dynamic fire suppression environment, a 400-millisecond delay results in the robot overshooting its target or colliding with an obstacle.

Solution (Edge Localization): This debugging revelation directly motivated the architectural migration to the local Mosquitto broker. By confining all publish/subscribe traffic entirely within the local area network subnet, external routing latency was eliminated, resulting in a stable, deterministic kill-switch response time regardless of external internet conditions.

11. Chapter 11: Verification & Results

11.1 Integration Testing Methodology

To validate the reliability, safety, and latency of the software integration architecture, a series of controlled unit and integration tests were conducted. Because the integration layer serves as the communication bridge for the entire Project Phoenix platform, testing focused heavily on network stability, multi-threading concurrency under load, and the deterministic execution of the kill-switch.

11.2 Network Latency and Broker Stability Results

The migration from a cloud-based broker to the local edge Mosquitto deployment yielded highly measurable performance improvements.

- **Cloud Broker (Previous):** Round-trip message latency averaged 120 milliseconds, with severe spikes up to 450 milliseconds during peak university network traffic. Disconnections occurred frequently.
- **Local Edge Broker (Current):** Round-trip message latency was reduced to a consistent 2 to 5 milliseconds over the local Wi-Fi subnet.

The localized Mosquitto broker demonstrated 100% uptime during all continuous testing phases, successfully processing parallel publish and subscribe requests from the Vision Node, the Flask microserver, and the Nav2 ROS 2 nodes without packet loss.

11.3 Multi-Threading and Video Telemetry Performance

The custom four-thread daemon architecture successfully resolved the procedural blocking issues observed in early prototypes. Under maximum system load—simultaneously running the ROS 2 navigation stack, capturing CSI camera frames, and serving HTTP requests—the backend integration layer remained highly stable.

The Flask streaming pipeline, utilizing the multipart/x-mixed-replace MIME type, successfully maintained two simultaneous MJPEG video feeds. The custom "Frame Drainage Engine" effectively prevented OpenCV buffer saturation. The resulting visual telemetry displayed a stable frame rate matching the camera hardware limits, with visual latency to the operator dashboard remaining under 15 milliseconds, providing true real-time situational awareness.

11.4 Kill-Switch Determinism

The most critical verification metric was the reliability of the dual-layer safety loop. The system was subjected to simulated fire detection events published directly to the robot/fire_detected topic.

- **Software Layer:** In 100% of test cases, the assertion of the fire_detected_flag instantly gated the /move/* REST API endpoints, successfully returning HTTP 403 Forbidden statuses to any incoming manual driving attempts
- **Hardware Layer:** Oscilloscope readings confirmed that upon receiving the alert payload, the duty cycle on the GPIO PWM pins dropped to 0% within less than 10 milliseconds, electrically disabling the BTS7960 motor drivers. The robot successfully transitioned from a state of active locomotion to absolute immobility instantaneously, satisfying the core safety requirement for hazardous deployment.

12. Chapter 12: Limitations & Future Work

12.1 Current System Limitations

While the software integration architecture successfully fulfilled the operational requirements of Project Phoenix, several limitations remain, primarily dictated by institutional safety guidelines, hardware constraints, and the inherent challenges of split-node computing.

1. **Absence of Live-Fire Environmental Testing:** Due to strict university safety regulations, full-scale system testing incorporating live, uncontrolled fire scenarios was restricted. System validation was performed using controlled thermal anomalies and digital simulations. Consequently, the impact of extreme environmental variables—such as dense smoke occluding the Pi Camera V3, or severe ambient heat inducing premature hardware throttling on the Raspberry Pi—remains unquantified in a real-world industrial fire.
2. **Network Decoupling Risks:** Although the local Mosquitto edge broker eliminated internet dependency, the split-node architecture still requires a stable local Wi-Fi connection between the Laptop Vision Node and the Mobile Edge Node. If physical interference (e.g., concrete warehouse walls or electromagnetic interference) entirely drops the local subnet, the robot will safely halt due to a timeout in the control loops, but it will lose its ability to receive new navigational coordinates.
3. **Computer Vision Boundary Conditions:** The integration layer relies heavily on the accuracy of the external Computer Vision models. [PLACEHOLDER-3: Briefly mention any known limitations from your CV teammates regarding the Human Detection or Fire Detection models, such as false-positive rates under specific lighting conditions or maximum detection range].

12.2 Future Architectural Enhancements

The backend API and MQTT architecture were specifically designed to be modular, allowing future engineers to seamlessly attach new capabilities to the platform without breaking existing safety loops. The primary areas targeted for future development include:

- 1. Real-Time SLAM Dashboard Integration:** Currently, the Flask microserver streams video telemetry and hardware metrics. Future iterations will focus on bridging the ROS 2 navigation stack directly into the web interface by serializing the Nav2 2D occupancy grid (SLAM map). This would allow remote operators to view the robot's real-time position mapped against detected warehouse obstacles.
- 2. API Security and Authentication:** For the scope of this prototype, the Flask control endpoints and MQTT broker utilize open local routing to facilitate rapid testing. Before industrial deployment, the backend must be hardened. This involves implementing JSON Web Token (JWT) authentication on all HTTP POST requests and enforcing TLS certificate verification on the Mosquitto broker to prevent unauthorized network hijacking.
- 3. Intelligent Alert Tiering:** The current kill-switch logic operates on a strict Boolean threshold (e.g., `fire_detected` is either True or False). A future enhancement to the API will introduce a tiered severity system. By parsing varying confidence levels and bounding-box sizes from the Vision Node, the integration layer could allow the robot to react differently to minor smoke (triggering an alert but continuing patrol) versus structural flames (triggering the immediate hardware kill-switch and suppression pump).

13. Conclusion

13.1 Project Summary and Architectural Success

The development of the Project Phoenix Intelligent Mobile Robot required overcoming significant barriers in embedded computing, asynchronous network communication, and fault-tolerant system design. The primary objective of this thesis was to engineer a robust software integration layer capable of bridging high-level artificial intelligence with low-level robotic actuation, without compromising the real-time responsiveness required in hazardous environments.

By implementing a distributed, split-node edge computing architecture, the platform successfully offloaded heavy Convolutional Neural Network processing to an external Vision Node, preserving Raspberry Pi 4's limited computational resources for critical ROS 2 navigation and safety-loop execution.

13.2 Key Engineering Achievements

The success of this integration was driven by several deliberate backend engineering decisions:

1. **Network Localization:** Migrating the communication infrastructure from a cloud-based dependency to a localized Mosquitto MQTT edge broker was a defining improvement. This transition eliminated external single points of failure, guaranteed absolute internet independence, and reduced message latency to sub-millisecond ranges, ensuring deterministic control.
2. **Asynchronous Microserver Design:** The development of a lightweight, multi-threaded Flask microserver provided a highly responsive programmatic interface. By decoupling the video telemetry pipelines, MQTT background listening, and HTTP request routing into independent daemon threads, the system eliminated procedural blocking. This ensured continuous visual telemetry delivery via MJPEG multipart streaming without stalling critical motor execution.
3. **Deterministic Fail-Safes:** The engineering of the dual-layer kill-switch mechanism proved highly effective. By combining software-level API routing restrictions with hardware-level PWM severing on the RPi .GPIO pins, the integration architecture guarantees absolute platform immobility the instant a fire anomaly is detected, satisfying strict operational safety requirements.

4. **Hardware Observability:** The proactive integration of the `psutil` and `vcgencmd` polling pipelines provided critical real-time visibility into the edge node's thermal and computational state, preventing silent thermal throttling and system desynchronization during deployment.

13.3 Final Remarks

In conclusion, the system integration, communication protocols, and teleoperation backend developed for Project Phoenix successfully unify disparate mechanical, electrical, and perceptual subsystems into a cohesive, autonomous entity. The decoupled, scalable architecture engineered in this work not only satisfies the immediate requirements of automated flame location detection, but also establishes a highly stable foundation for the future development of industrial-grade emergency response robotics.

Reference

- [1] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Datasheet," *Raspberry Pi Documentation*, 2023. [Online]. Available: <https://www.raspberrypi.com/documentation/>
- [2] Lightstone, R., *Eclipse Mosquitto: An open source MQTT broker*, Eclipse Foundation, 2022. [Online]. Available: <https://mosquitto.org/>
- [3] Macenski, S., et al., "The Marathon 2: A Navigation System," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. (ROS 2 Nav2 Architecture).
- [4] Pallets Projects, "Flask Documentation (Version 3.0.x)," *The Pallets Projects*, 2023. [Online]. Available: <https://flask.palletsprojects.com/>
- [5] Infineon Technologies, "BTS7960 High Current PN Half Bridge NovalithIC," *Infineon Datasheets*, Rev. 1.1, 2004.
- [6] Bradski, G., "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000. [Online]. Available: <https://opencv.org/>
- [7] TP-Link Technologies, "Tapo C210 Pan/Tilt Home Security Wi-Fi Camera Specification," *TP-Link Product Documentation*, 2023.
- [8] Banks, A., Gupta, R., "MQTT Version 3.1.1," *OASIS Standard*, 2014. [Online]. Available: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>

Appendix A: MQTT Topic & JSON Payload Schemas

The following JSON schemas define the exact data structures utilized by the backend integration layer over the local Mosquitto broker.

A.1 Safety Kill-Switch Trigger (robot/fire_detected) Published by the Global Vision Node to trigger the dual-layer deterministic kill-switch on the Edge Node.

```
{  
  "fire": true,  
  "confidence": 0.8753,  
  "timestamp": "2026-06-13T10:23:45Z"  
}
```

A.2 Navigational Coordinate Dispatch (ambers/robot/navigation/target) Published by the Global Vision Node to initiate autonomous ROS 2 Nav2 locomotion.

```
{  
  "x": 4.25,  
  "y": -1.15  
}
```

A.3 Real-Time Hardware Telemetry (/status HTTP GET Endpoint) Generated every two seconds by the psutil daemon thread to populate the remote operator dashboard.

```
{  
  "cpu_temp_c": 62.5,  
  "cpu_usage_percent": 45.2,  
  "ram_used_mb": 1450,  
  "ram_total_mb": 3840,  
  "safety_lockout_active": false,  
  "camera_stream_alive": true  
}
```

Appendix B: Edge Node Hardware Pin Mapping

The following table details the physical hardware connections between the Raspberry Pi 4 (Mobile Edge Node) and the robotic actuation peripherals, specifically isolated by the Motor Control Thread.

Table B.1: Broadcom (BCM) GPIO Allocations

Peripheral Component	Pin Function	Pi Physical Pin	BCM GPIO Pin	Logic Level
Right BTS7960 Driver	Forward PWM (R_EN)	Pin 11	GPIO 17	3.3V (Variable)
Right BTS7960 Driver	Reverse PWM (L_EN)	Pin 13	GPIO 27	3.3V (Variable)
Left BTS7960 Driver	Forward PWM (R_EN)	Pin 16	GPIO 23	3.3V (Variable)
Left BTS7960 Driver	Reverse PWM (L_EN)	Pin 22	GPIO 25	3.3V (Variable)
Suppression Pump Relay	Trigger Signal (IN)	Pin 37	GPIO 26	3.3V (Active High)
System Ground	Common Ground	Pin 39	GND	0V